PRML Lab Assignment 7

Gaussian Mixture Models and Pattern Recognition

Karthikeyan P L
Registration Number: 26120031
Team 16

October 2026

1 Introduction

Pattern recognition involves assigning an observed sample to one of several possible classes based on its measured features. A central challenge is that observations belonging to the same class rarely form a single compact cluster. Variations in shape, orientation, scale, and other factors may create several subgroups or modes within one class. A single Gaussian distribution can be restrictive when the underlying data distribution is multimodal.

A Gaussian Mixture Model (GMM) addresses this limitation by representing a probability density as a weighted sum of Gaussian components. Each component captures one local region of the distribution, while the mixture combines these regions into a flexible overall model. In this assignment, a separate GMM is learned for each class and classification is performed by comparing class-conditional likelihoods.

The experiment has two parts. First, the synthetic two-class data generated in Assignment 6 is reused to build class-wise GMMs and evaluate their classification performance. Second, GMMs are trained for the assigned MNIST handwritten digits 3, 5, and 9. The latter experiment includes dimensionality reduction, model selection using a development set, test-set evaluation, and interpretation of misclassified samples.

2 Theoretical Background

2.1 Gaussian probability density

For a d-dimensional feature vector $\mathbf{x}$, a multivariate Gaussian distribution is defined by mean $\boldsymbol{\mu} \in \mathbb{R}^d$ and covariance matrix $\boldsymbol{\Sigma} \in \mathbb{R}^{d\times d}$:

$$\mathcal{N}(\mathbf{x};\boldsymbol{\mu},\boldsymbol{\Sigma}) = \frac{1}{(2\pi)^{d/2}|\boldsymbol{\Sigma}|^{1/2}} \exp\left[-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^T\boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu})\right].$$

The mean specifies the center of the distribution. The covariance describes spread and correlation between dimensions. The determinant $|\boldsymbol{\Sigma}|$ controls the volume scaling, while the quadratic form in the exponent measures squared Mahalanobis distance from the mean.

2.2 Gaussian Mixture Model

A K-component GMM is a convex combination of Gaussian densities:

$$p(\mathbf{x} \mid \theta) = \sum_{k=1}^{K} \pi_k \mathcal{N}(\mathbf{x};\boldsymbol{\mu}_k,\boldsymbol{\Sigma}_k), \qquad \pi_k \geq 0, \quad \sum_{k=1}^{K}\pi_k = 1.$$

Here $\theta = \{\pi_k, \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k\}_{k=1}^{K}$ denotes the model parameters. The mixture weight π_k is the prior probability of selecting component k. The component means and covariance matrices describe the

location and shape of local clusters. Unlike hard clustering, a GMM allows a point to belong probabilistically to multiple components.

2.3 Class-conditional density estimation and decision rule

For a classification problem with classes $c \in \mathcal{C}$, one model $p(\mathbf{x} \mid \theta_c)$ is trained for each class. The maximum-likelihood classifier is

$$\hat{c}(\mathbf{x}) = \arg\max_{c \in \mathcal{C}} p(\mathbf{x} \mid \theta_c) = \arg\max_{c \in \mathcal{C}} \log p(\mathbf{x} \mid \theta_c).$$

The logarithm is used because likelihoods may be extremely small; it improves numerical stability and converts products into sums. If class priors are unequal and known, a Bayesian classifier can instead maximize $\log p(\mathbf{x} \mid \theta_c) + \log P(c)$. In the present experiments, classification is based on the class-model likelihoods.

2.4 Parameter estimation using Expectation-Maximization

Direct maximum-likelihood estimation for a mixture is difficult because the component identity responsible for each observation is latent. The Expectation-Maximization (EM) algorithm iteratively estimates these hidden assignments and updates the parameters.

E-step: responsibilities. For observation $\mathbf{x}_n$, the posterior responsibility of component k is

$$\gamma_{nk} = P(z_n = k \mid \mathbf{x}_n) = \frac{\pi_k \mathcal{N}(\mathbf{x}_n; \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{\sum_{j=1}^{K} \pi_j \mathcal{N}(\mathbf{x}_n; \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)}.$$

Thus γ_{nk} expresses how strongly component k explains sample n and satisfies $\sum_k \gamma_{nk} = 1$.

M-step: parameter updates. Define effective membership $N_k = \sum_{n=1}^{N} \gamma_{nk}$. The updated parameters are

$$\pi_k^{\text{new}} = \frac{N_k}{N}, \qquad \boldsymbol{\mu}_k^{\text{new}} = \frac{1}{N_k} \sum_{n=1}^{N} \gamma_{nk} \mathbf{x}_n,$$

$$\boldsymbol{\Sigma}_k^{\text{new}} = \frac{1}{N_k} \sum_{n=1}^{N} \gamma_{nk} (\mathbf{x}_n - \boldsymbol{\mu}_k^{\text{new}})(\mathbf{x}_n - \boldsymbol{\mu}_k^{\text{new}})^T.$$

The E-step and M-step are repeated until convergence or a maximum iteration limit is reached. Each iteration is designed to not decrease the data log-likelihood, although practical implementations may add regularization to prevent singular covariance estimates.

2.5 Initialization and model complexity

EM is sensitive to its initial values and may converge to a local optimum. K-means clustering is commonly used to initialize component means; initial covariances are estimated from assigned samples and weights from cluster proportions. Multiple initializations can be tried and the best-fitting solution retained.

The number of components K controls model flexibility. A small K may underfit a complex distribution, while a very large K may overfit and increase computational cost. In this experiment, candidate values are evaluated on a development set; the test set is held out until final evaluation to avoid optimistic bias.

3 Question 1: GMM classification on Assignment 6 synthetic data

3.1 Data generation and preparation

The two-class dataset from Assignment 6 was regenerated using the same custom random-generation procedure. Each class contains 5,000 two-dimensional samples formed from three Gaussian components. The class-1 mixture weights are $(0.40, 0.35, 0.25)$ and class-2 weights are $(0.30, 0.40, 0.30)$. The isotropic covariance setting was used for this run. Each class was split into 80% training and 20% test samples, giving 4,000 training and 1,000 test observations per class.

The class-wise split ensures that both classes have representation in training and testing. The training partition is used only to estimate the distributions; the test partition provides an independent measure of generalization.

3.2 Model fitting

A three-component full-covariance GMM was fitted independently to the training samples of each class. The component parameters were initialized and refined using EM. Both class models converged in three EM iterations in this run. For an observation $\mathbf{x}$, the predicted label is

$$\hat{c}(\mathbf{x}) = \arg\max_{c\in\{1,2\}} \log\left[\sum_{k=1}^{3} \pi_{ck}\mathcal{N}(\mathbf{x};\boldsymbol{\mu}_{ck},\boldsymbol{\Sigma}_{ck})\right].$$

3.3 Results

The test accuracy was **97.60%**, corresponding to 1,952 correct classifications out of 2,000 test samples. The confusion matrix is shown below.

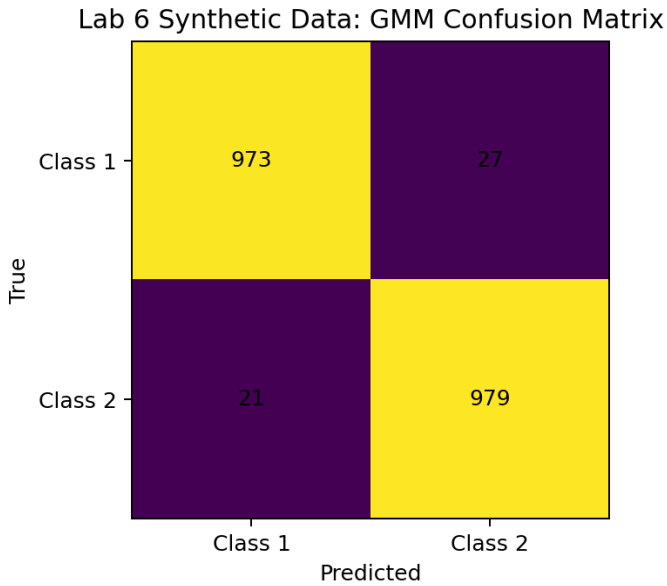


Figure 1: Q1 confusion matrix (rows indicate true class; columns indicate predicted class).

Table 1: Q1 synthetic-data test confusion matrix

	Predicted Class 1	Predicted Class 2
True Class 1	973	27
True Class 2	21	979

3.4 Interpretation

The diagonal entries (973 and 979) represent correct predictions, while 27 class-1 points and 21 class-2 points were assigned to the opposite class. The total of 48 errors is small compared with the 2,000 test points, indicating that the learned class densities describe the generated data effectively. Remaining errors are expected near regions where the two class distributions overlap: in such regions, a sample may have comparable likelihood under both GMMs, and small parameter differences can reverse the decision.

4 Question 2: GMM models for MNIST digits 3, 5, and 9

4.1 Dataset and image representation

The assigned classes are handwritten digits 3, 5, and 9. Each MNIST image is grayscale and has spatial dimensions 28×28. It is flattened into a feature vector of length 784 and pixel values are normalized by dividing by 255, placing intensities in the range $[0, 1]$.

The data partitions used in the experiment are summarized below. The training split is used to fit parameters, the development split is used to select the number of mixture components, and the test split is reserved for final evaluation.

Table 2: MNIST sample counts for the assigned digits

Partition	Digit 3	Digit 5	Digit 9
Training	4,886	4,368	4,776
Development	1,245	1,053	1,173
Test	1,010	892	1,009

4.2 Principal Component Analysis

Directly fitting GMMs in 784 dimensions is computationally demanding and may be unstable when covariance matrices are estimated from finite data. Principal Component Analysis (PCA) maps the data to a lower-dimensional orthogonal coordinate system while retaining directions with high variance.

Let X be the centered training matrix. Its sample covariance is

$$S = \frac{1}{N-1} X^T X.$$

Eigen-decomposition yields $S\mathbf{v}_i = \lambda_i \mathbf{v}_i$, where eigenvectors are principal directions and eigenvalues indicate variance along those directions. Eigenvalues are sorted in descending order and the first d' eigenvectors form projection matrix W. A sample is projected as $\mathbf{z} = W^T(\mathbf{x} - \boldsymbol{\mu})$.

PCA was computed using covariance eigendecomposition (‘numpy.linalg.eigh‘). Retaining 88 principal components reduced the dimension from 784 to 88 while preserving **90.70%** of total variance. The transformation was learned from training data and then applied consistently to development and test data.

4.3 Selection of number of components

A separate GMM was trained for each digit. The candidate component counts $K = 1, 2, \ldots, 9$ were compared using development accuracy. The observed accuracies are:

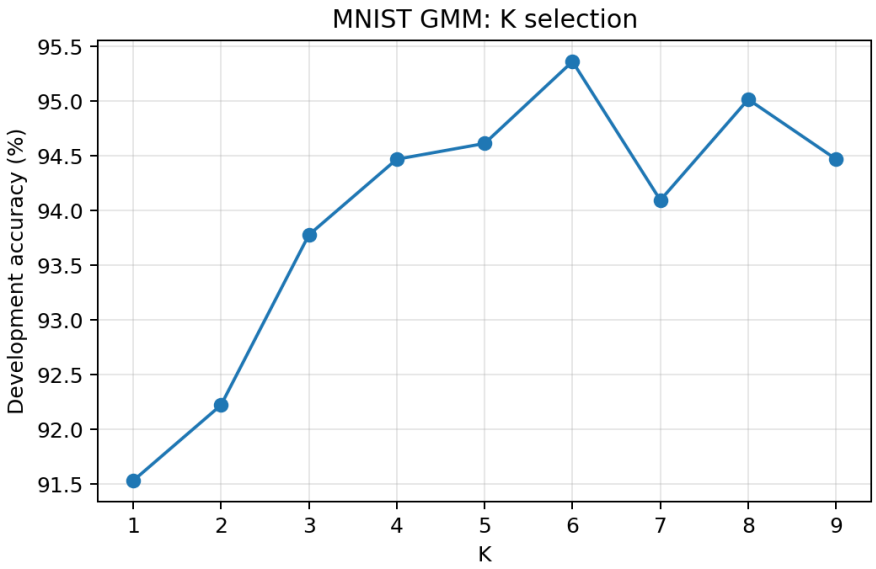


Figure 2: Development accuracy for candidate mixture-component counts.

Table 3: Development-set accuracy by number of components

K	1	2	3	4	5	6	7	8	9
Accuracy (%)	91.53	92.22	93.78	94.47	94.61	**95.36**	94.09	95.02	94.47

The selected configuration was $K = 6$ components per digit. Although $K = 8$ also performs well, $K = 6$ gives the highest recorded development accuracy (95.36%) among the tested values. This selection is based on development data rather than test accuracy, keeping the final evaluation unbiased.

4.4 Final model training

With the dimensionality-reduction setting and component count selected, class-wise GMMs were fitted to the training data for digits 3, 5, and 9. Each model estimates its own mixture weights, component means, and covariance matrices. For a test image $\mathbf{x}$, the model computes three class-conditional log-likelihoods and assigns the digit with the largest value.

5 Question 3: Test-set classification and evaluation

5.1 Decision rule

For the three classes, classification is performed as

$$\hat{c}(\mathbf{x}) = \arg \max_{c \in \{3,5,9\}} \log p(\mathbf{x} \mid \hat{\theta}_c).$$

The evaluation uses the held-out test images only once after the model configuration has been selected. The confusion matrix convention is rows = true digits and columns = predicted digits, ordered as 3, 5, and 9.

5.2 Overall performance

The final test accuracy was **95.7746%**. Out of 2,911 test images, 2,788 were classified correctly and 123 were misclassified.

Table 4: MNIST test confusion matrix (true labels by predicted labels)

	Predicted 3	Predicted 5	Predicted 9
True 3	966	33	11
True 5	30	848	14
True 9	13	22	974

Table 5: Per-class precision, recall, and F1-score

Digit	Precision	Recall	F1-score
3	0.96	0.96	0.96
5	0.94	0.95	0.94
9	0.97	0.97	0.97

Precision measures the fraction of samples predicted as a digit that truly belong to it; recall measures the fraction of actual samples of that digit correctly identified. The F1-score is the harmonic mean of precision and recall. Digit 9 has the strongest rounded class-wise scores, while digit 5 is comparatively more difficult, consistent with its greater off-diagonal counts.

6 Question 4: Analysis of digit confusions using GMMs

6.1 Observed error patterns

The largest off-diagonal counts are summarized below. The chart provides a compact visual summary of all six directed error types.

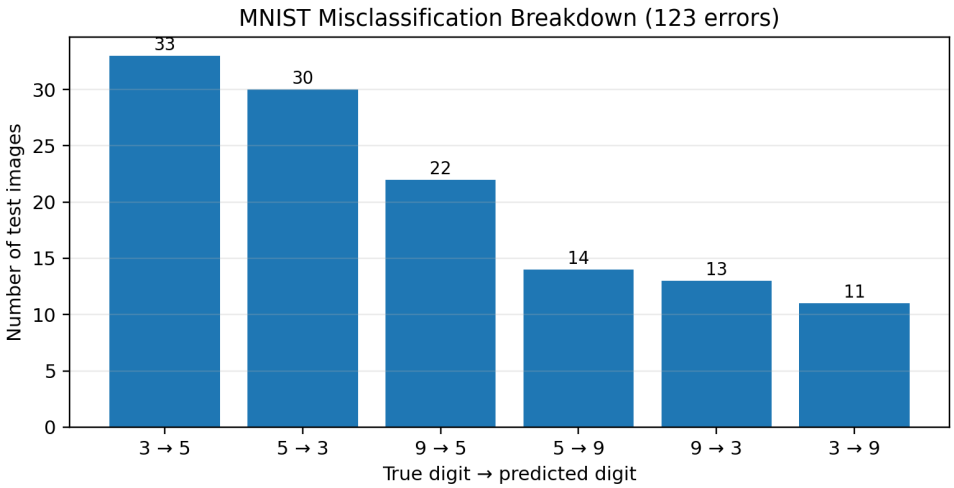


Figure 3: Counts of test images assigned to an incorrect digit class.

- Digit 3 → digit 5: 33 images; digit 5 → digit 3: 30 images.
- Digit 9 → digit 5: 22 images; digit 5 → digit 9: 14 images.
- Digit 3 → digit 9: 11 images; digit 9 → digit 3: 13 images.

The most prominent confusion is between digits 3 and 5. These handwritten forms can share curved strokes, open loops, and similar upper or lower contours. Some examples of 9 may also resemble 3 or 5 when the loop is weakly closed or the descending stroke is atypical.

6.2 Interpretation through class-density boundaries

A GMM decision boundary between classes a and b consists of points satisfying

$$\log p(\mathbf{x} \mid \hat{\theta}_a) = \log p(\mathbf{x} \mid \hat{\theta}_b).$$

On one side of this boundary, class a has higher likelihood; on the other, class b does. Misclassified samples often lie close to such boundaries or in regions where the estimated densities overlap. Because each digit model has several Gaussian components, it can represent different writing styles; however, the mixture components are still smooth density approximations and cannot perfectly separate ambiguous handwriting.

The 3-to-5 and 5-to-3 errors are consistent with overlapping regions in feature space. Likewise, 9-to-5 errors may occur for samples whose loop and tail do not strongly match the dominant patterns represented by the digit-9 model. These observations illustrate that classification is not based on a single prototype: it compares the complete mixture likelihood, including all components and their weights.

6.3 What the learned component means represent

The mean of each Gaussian component can be reshaped from the 88-dimensional PCA representation back through the PCA projection to visualize a representative image-like pattern. Components within the same digit capture different writing styles, stroke thicknesses, slants, or local shape variations. The collection of means therefore gives a qualitative view of how the GMM partitions the within-class variability.

7 Conclusion

This assignment demonstrated probabilistic classification using class-conditional Gaussian Mixture Models. In the synthetic experiment, three-component GMMs trained with EM achieved 97.60% test accuracy on the two-class data. For MNIST digits 3, 5, and 9, PCA reduced the feature dimension from 784 to 88 while retaining 90.70% variance. Development-set model selection favored six components per digit, and the final test accuracy was 95.7746%.

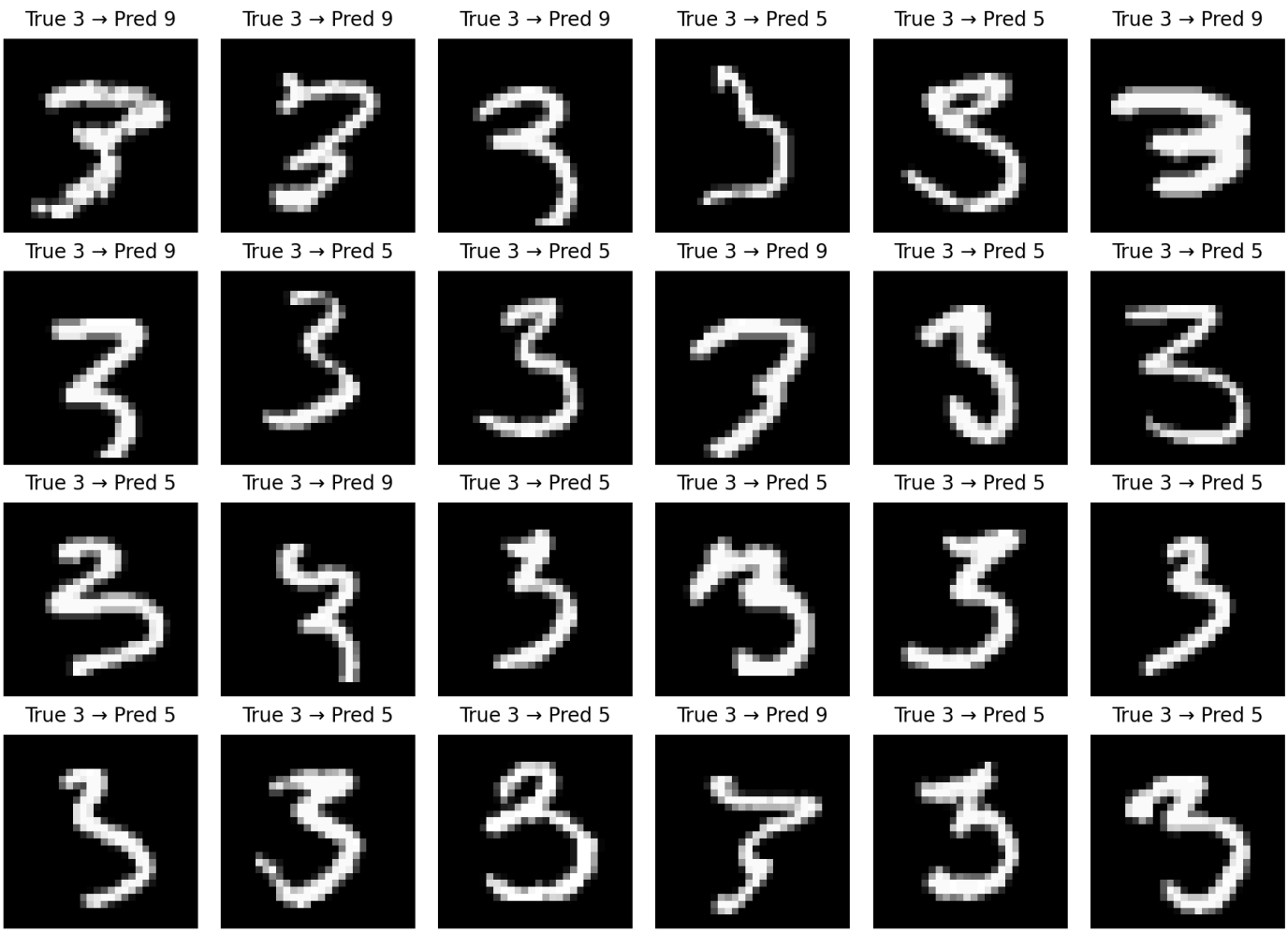
The confusion matrix and error analysis reveal that most mistakes occur between visually similar handwritten digits, especially 3 and 5. The experiment reinforces the key advantage of GMMs: they can represent multimodal class distributions using multiple Gaussian components. Their performance nevertheless depends on adequate representation, initialization, covariance estimation, and overlap between class densities.

Reproducibility and implementation notes

For the synthetic-data experiment, the Assignment 6 isotropic generation procedure was rerun with the stated mixture parameters and a fixed split seed. Three full-covariance components were fitted per class using EM, with five initializations and a maximum of 100 iterations. The MNIST results, partition counts, PCA dimension, component-selection accuracies, and final confusion matrix are reported from the Lab 7 experiment. The development set was used for selecting K; the test set was kept separate for final reporting.

MISCLASSIFIED MNIST TEST EXAMPLES

Actual images from TEAM_16/test; labels show true digit · predicted digit.



The montage displays 24 representative errors from the 129 misclassified test samples.

Assignment Questions (Original Problem Statement)

1. Using the data generated in Assignment 6, build GMM models for each of the classes using 80% of the data, and perform classification. Create the confusion matrix.

2. Use the MNIST dataset available on the website. The MNIST dataset consists of images of handwritten characters corresponding to digits 0–9. Each group will work with 3 different digits. Treat each digit as an image. Each normalized image is made up of 784 pixel values. There are about 60000 images in the training set. Use the 3 digits assigned to your team. You could further compress the 784 pixel values to represent them as image patches. Treat each image or image patch as training data. Build a GMM model for each digit using ONLY the training data.

3. Using the model trained as above, perform the classification on the test data. Record your results.

4. When digits are confused, you must try to explain it with the GMM model trained – primarily look at the boundaries between classes.